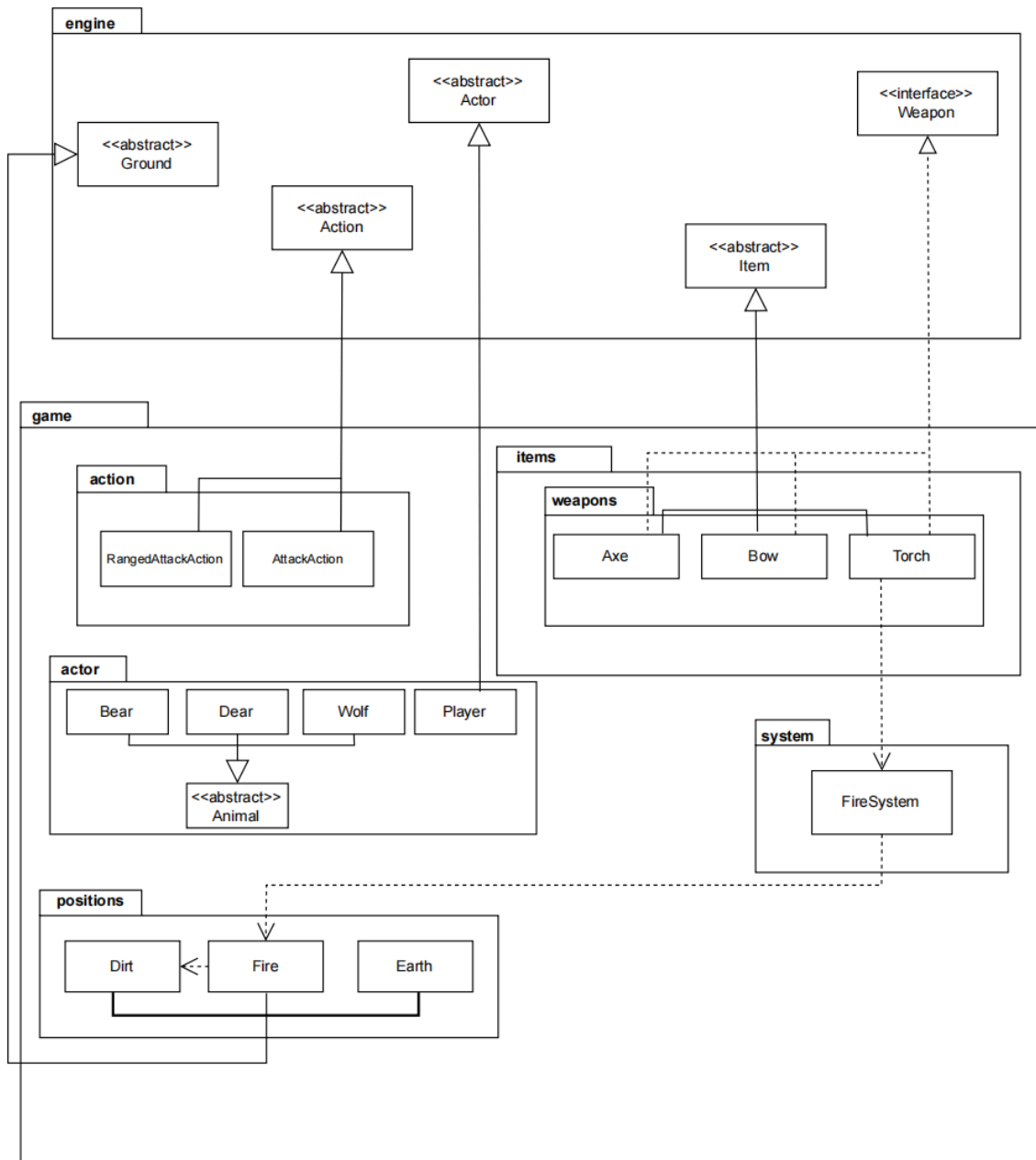


REQ 3 UML Class Diagram:



Design Rationale

REQ 3

A. Weapon Representation

Design 1: Create an abstract class AbstractCoatableWeapon that extend IntrinsicWeapon. Weapons such as Axe and Bow extend AbstractCoatableWeapon.

Pros	Cons
IntrinsicWeapon already implements attack() using damage and hit rate, so Axe automatically gets all that behavior without needing to reimplement it.	Conceptual mismatch Axe should be an item, not intrinsic. IntrinsicWeapon represents <i>natural, unremovable body parts</i> (like claws, fangs, fists). So making Axe extend IntrinsicWeapon is semantically incorrect, it breaks your object model.
If you just need a working weapon for testing attack balance or hit rates, this shortcut works fine.	

Design 2: Create an abstract class AbstractCoatableWeapon that extend Item and implement Weapon. Weapons such as Axe and Bow extend AbstractCoatableWeapon.

Pros	Cons
Fully compatible with the existing item system (can be picked up, dropped, or stored).	Mixing items and weapons in the same hierarchy may require careful documentation to clarify responsibilities.
Clean integration with the engine's Actor and Action system — no need to modify the engine.	
Simple, lightweight design that aligns with existing game structure.	

B. Status Effects Management

Design 1: Instead of a single static StatusEffects manager, each actor could own its own EffectManager instance. This manager holds a list or map of active effects specific to that

actor. All effects implement a shared interface (e.g. `StatusEffect`) that defines lifecycle methods such as `onApply`, `onTurn`, and `onRemove`. The game loop or actor's `tick()` method calls the actor's `EffectManager.updateEffects()`, which processes and removes expired effects. Each effect can interact directly with the actor, without a centralized static registry.

Pros	Cons
Each actor fully owns and manages its effects, reducing global dependencies. This makes the system more self-contained and aligns with OOP principles, where objects handle their own behavior.	Each actor needs its own effect manager, which slightly increases memory usage. In large games with many small entities, this can add up, though it's typically negligible.
Having per-actor managers allows independent worlds, simulations, or threads without interference. It becomes easier to extend the system for multiplayer or multiple game instances, where global static state would cause issues.	You must ensure every actor initializes and updates its manager correctly. If this setup is forgotten for some subclass, its effects won't work — unlike a global manager that covers all actors automatically.
Because each actor's effects are localized, unit testing becomes easier. You can test an actor's behavior with specific effects without needing to initialize or mock a global manager	Each actor must call its effect manager every turn or tick. If the update loop isn't properly synchronized, some actors might miss effect updates or persist expired effects too long.

Design 2: The `StatusEffects` class acts as a global manager that tracks and updates temporary conditions applied to actors, such as bleed, burn, poison, and frostbite. It uses a `WeakHashMap` to associate each actor with a list of active effects, ensuring automatic cleanup when actors are removed. Each effect is implemented as a small, self-contained class that defines how it behaves each turn and when it expires.

Pros	Cons
All status effects are handled in one place, simplifying updates and maintenance. You can easily process or remove all effects for an actor without scattering logic across multiple classes.	Damage-over-time effects like bleed, burn, and poison share similar logic (damage per turn, duration tracking, printing messages). Without an abstract superclass or shared utility, this can lead to code duplication and harder maintenance.
The use of <code>WeakHashMap</code> ensures that when an actor is no longer in use, its associated effects are automatically garbage-collected. This prevents memory leaks without needing explicit	Frostbite's behavior depends on manually checking the actor type (like <code>instanceof</code>), which breaks polymorphism. A more scalable approach would use a unified stats or

cleanup code.	attributes system to adjust properties like speed or attack rate.
Each effect is encapsulated in its own class, following a clear interface (apply, update, remove, etc.). This makes it easy to add new effects or tweak existing ones without changing the rest of the system.	The StatusEffects class is static, meaning there's only one global instance. This design works for small games but becomes restrictive if you introduce multiple worlds, threads, or game instances, as each would need its own independent effect system.

C: Fire System Integration

Design 1: Instead of using a static utility class, create a FireManager class that is responsible for handling all fire-related activities across the game. The FireManager would store active fire zones, manage their duration, and apply effects each turn.

This design centralizes all fire-related environmental effects into one place, allowing any actor or item to create fire without reimplementing the same logic. It is a procedural, utility-style design that focuses purely on effect execution rather than long-term management or configuration.

Pros	Cons
FireManager can track all active fires and handle them efficiently in bulk during the game's update phase, rather than scattering logic across many systems.	Managing active fires, lifetimes, and updates adds overhead and requires careful design to prevent memory leaks or update conflicts.
Because the system is managed, you can easily add features like spreading fire, extinguishing under rain, or merging fires without touching multiple weapon classes.	The game engine must call FireManager.tick() each turn or integrate it into its event system, which increases coupling with the main game loop.
Other systems (sound, visuals, achievements) can listen to fire events (like fireCreated or fireExtinguished), improving modularity.	Tracking and updating many fires each turn can slightly increase processing time if not optimized, especially on large maps.

Design 2: The FireSystem class is a static utility that handles spreading fire on the map. It has one main method, spawnAround, which takes an attacker, a map, a radius, and a fire lifetime value. When called, it finds all tiles within the given radius around the attacker (except the attacker's own tile) and changes their ground to a Fire object with a specified lifespan.

This design centralizes all fire-related environmental effects into one place, allowing any actor or item to create fire without reimplementing the same logic. It is a procedural, utility-style design that focuses purely on effect execution rather than long-term management or configuration.

Pros	Cons
Other systems can call <code>FireSystem.spawnAround</code> directly without worrying about how fire is created or applied. It makes code shorter and clearer.	Because it uses static methods, you cannot customize or extend its behavior without changing the class directly. For example, you cannot easily add different fire behaviors like magical fire or spreading fire.
All logic related to placing fire is in one place, making it easy to maintain and update. If you need to change how fire spreads, you only update this one class.	The system directly creates <code>Fire</code> objects, which ties it to one specific implementation. If the fire logic changes, you must modify <code>FireSystem</code> as well.
The same system can be used by different weapons, abilities, or events that need to spread fire, avoiding code duplication.	The method does not check whether the ground is flammable or already occupied. It blindly replaces whatever is there, which may cause unrealistic or buggy results.

D: Ranged Weapon

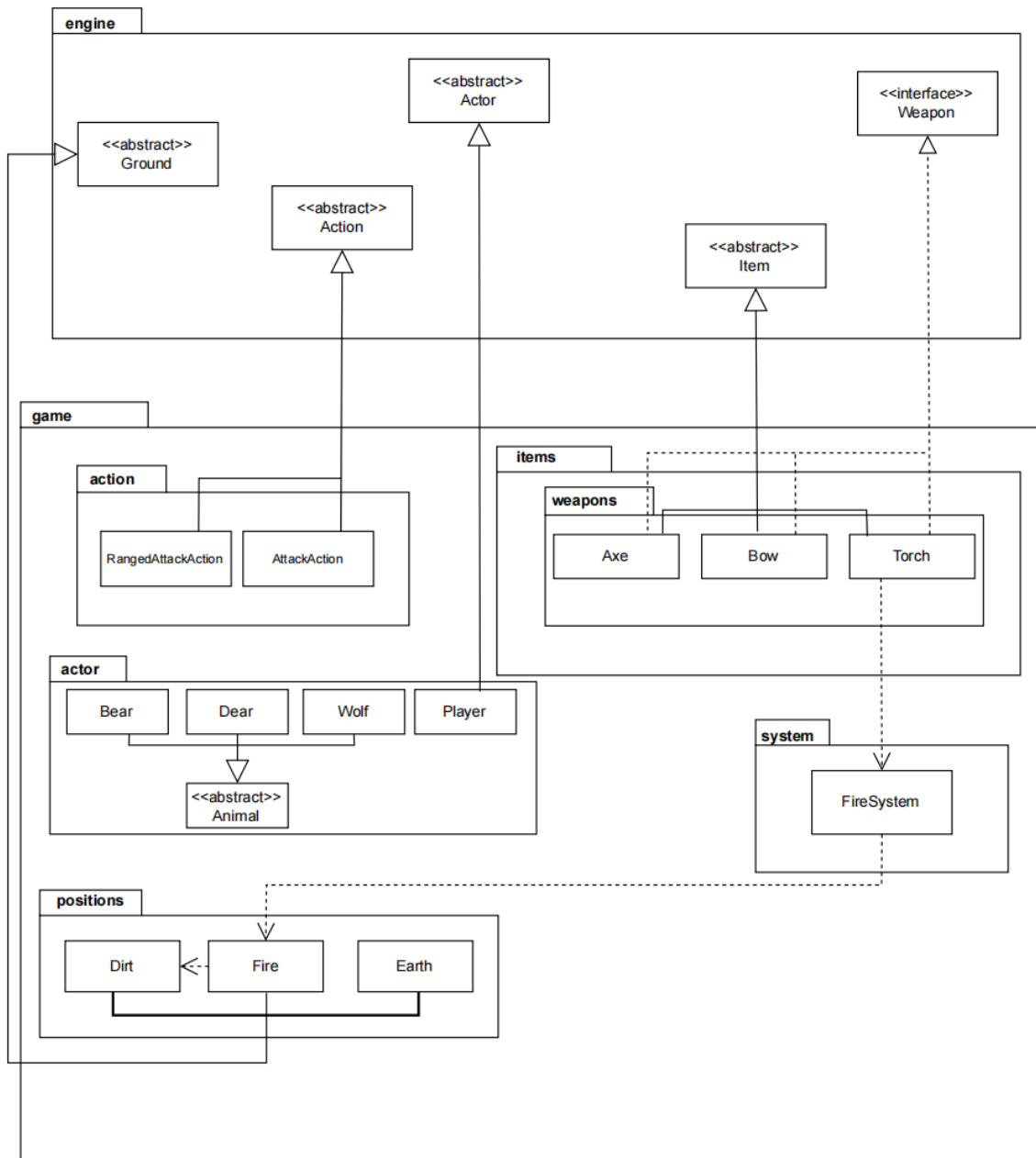
Design 1 : The `Bow` class directly handles all ranged attack logic — including distance checking, hit probability, and damage calculation — inside its own `attack()` method without using a separate class.

Pros	Cons
Simple and easy to implement since all logic is contained within one method.	Violates the Single Responsibility Principle because the <code>Bow</code> class handles both weapon logic and attack mechanics.
No need to create additional classes or files.	
Quick to prototype for small-scale testing.	Poor scalability — adding new ranged weapons would require rewriting similar distance and accuracy logic in each class.

Straightforward control flow as everything happens within one place.	Hard to maintain and debug since the ranged attack logic is tightly coupled to the Bow implementation.
	Breaks Open-Closed Principle , as any changes to ranged mechanics require modifying the Bow class directly.
	Difficult to reuse ranged attack logic for other weapons (e.g., a crossbow or magic staff).

Design 2 : The Bow allows the player to attack targets from up to 3 tiles away. It uses a dedicated RangedAttackAction class to handle distance checking and hit probability.

Pros	Cons
Clean separation between melee and ranged combat logic.	Requires additional logic to check line-of-sight and range.
Keeps ranged combat modular and reusable for future weapons.	
Improves code readability and follows the Single Responsibility Principle.	



The diagram above shows the final design of Requirement 3, which implements the new weapon system using the finalized design 2 from parts A, B, C, and D described above. This design introduces three concrete weapon classes — Axe, Torch, and Bow — each extending from the engine’s Item class and defining their own unique attack logic. The system also integrates a centralized StatusEffects manager for handling timed effects like Bleed and Burn, and reuses the existing FireSystem, Fire, and Dirt classes from earlier requirements to manage environmental fire tiles.

By separating each weapon into its own class, the design achieves modularity and adheres to the Single Responsibility Principle (SRP), as each weapon handles only its specific behavior and attack outcome.

The centralized StatusEffects class reduces code duplication and follows the DRY principle by managing all recurring damage-over-time effects in a single, consistent system.

This approach also supports the Open-Closed Principle (OCP), since new weapons or effects can be added without modifying existing code.

The reuse of FireSystem, Fire, and Dirt demonstrates effective integration with the existing game framework, avoiding redundant implementations while maintaining consistency with earlier requirements.

The design avoids any use of instanceof checks by relying on capabilities and composition, which keeps each component self-contained and aligned with KISS (Keep It Simple) and Interface Segregation principles.

Overall, the combination of modular weapon classes, centralized status management, and clean environmental integration results in a design that is robust, extensible, and maintainable.

It clearly demonstrates adherence to core software engineering principles such as SRP, DRY, OCP, and KISS, while ensuring smooth engine compatibility and future scalability.